FPGA でレンダリングを行う Wayland コンポジタ

2026-03-20 | Kernel/VM 探検隊@つくば No3

わたすけ(@watasuke1024)

わたすけ

watasuke

- **筑波大学**
  情報メディア創成学類 新 4 年
- **Twitter: watasuke1024**
  その他: watasuke102
- Web / 低レイヤ（自作 OS）
  - ❤ Rust / TypeScript / C++

目次

1. つくったもの
2. COJT とは
3. COJT でのあゆみ
4. 「自由課題」のあゆみ

目次

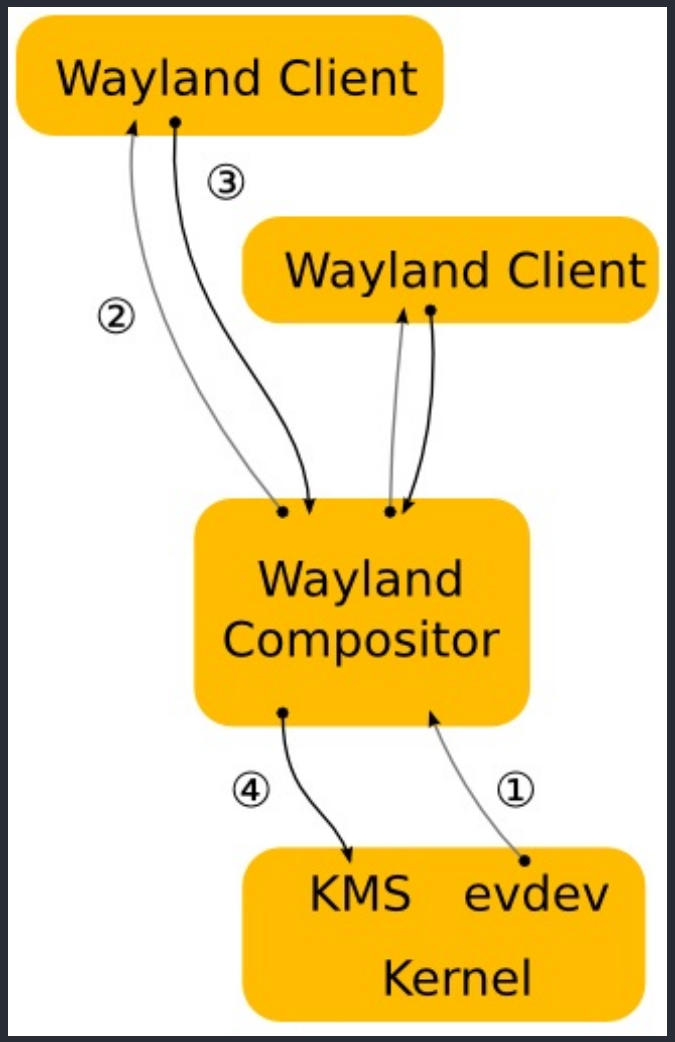
1. つくったもの
2. COJT とは
3. COJT でのあゆみ
4. 「自由課題」のあゆみ

つくったもの

- Linux が動く FPGA ボードの上で動く
  Wayland コンポジタ（ウィンドウマネージャ）
- 特徴：**FPGA が 自律的に描画**を行う
  - ループで render() を呼ぶなどして
    明示的にレンダリングを行わなくて良い

Wayland？

- GUI を実現する {ライブラリ,プロトコル}
- X Window System の後継という文脈で語られることが多い
- **Wayland コンポジタ**は、Wayland クライアントと **Wayland プロトコル**でやりとりをして、**ウィンドウを管理**する



Wayland のアーキテクチャ図

gitlab.freedesktop.org/wayland/wayland.freedesktop.org/-/blob/main/wayland-architecture.png

Wayland コンポジタ、すごい

いろいろやっている

- ウィンドウの座標・サイズを管理する**だけではない**
- 移動・リサイズ・最小化・最大化
- キーボード・マウスといった入力装置の管理
- ドラッグ & ドロップ
- コピー & ペースト

Q. なんで FPGA？

Q. なんで FPGA？

A. **COJT という授業の一環**で作ったから

目次

1. つくったもの
2. **COJT とは**
3. COJT でのあゆみ
4. 「自由課題」のあゆみ

COJT

- **組み込み技術キャンパス OJT**
- 筑波大学の情報学群で行われている授業
- ハードウェアコースとソフトウェアコースがある
  - FPGA をやるのは前者

COJT ハードウェアコース

- **FPGA で映像信号**を扱う
- VGA 信号を出力したり、
  コマンドを受け取って描画を行ったり……
- 高い機材やソフトウェアが使えて、うれしい
- いい感じの教室が **24 時間いつでも使い放題！**

目次

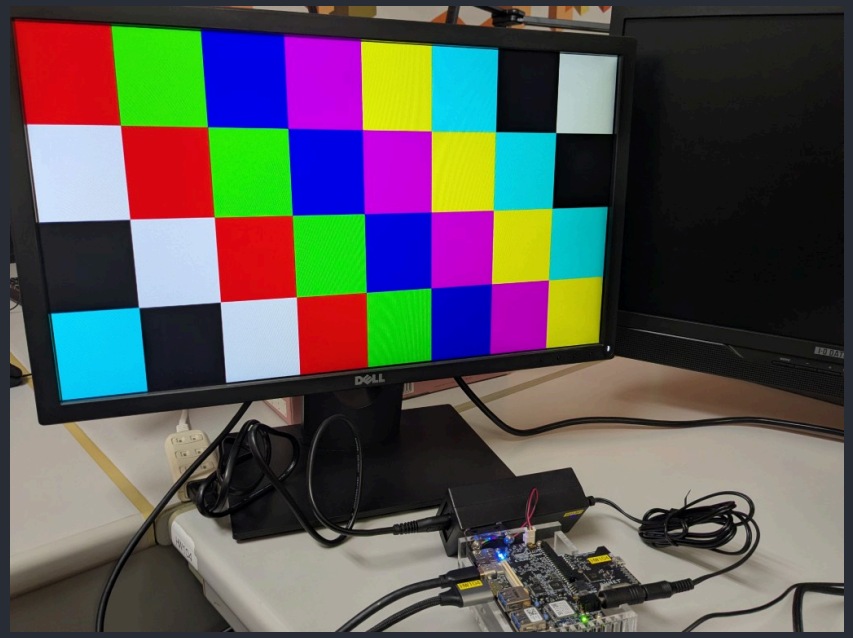
1. つくったもの
2. COJT とは
3. **COJT でのあゆみ**
4. 「自由課題」のあゆみ

大まかな流れ

1. パターン表示回路
2. 表示回路
3. フィルタ回路
4. 描画回路
5. 自由課題

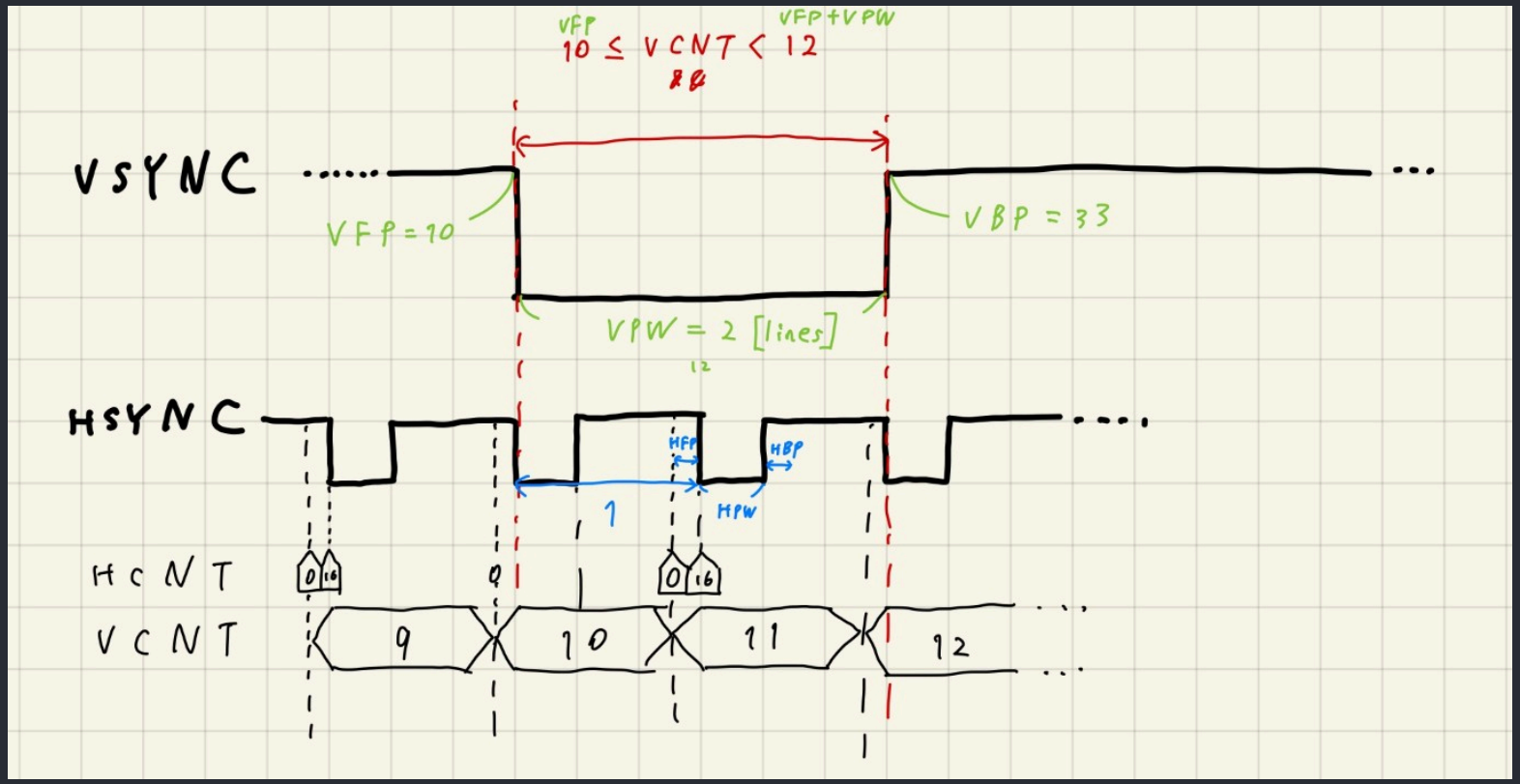
パターン表示回路

- カラーバーみたいなやつを表示
- 画像のような**パターンを生成する patgen と VGA 信号を生成する syncgen** の 2 つを主に実装



VGA 信号？

- {垂直,水平}同期信号 (VSYNC, HSYNC) を出力



実際に使っていたメモ

表示回路

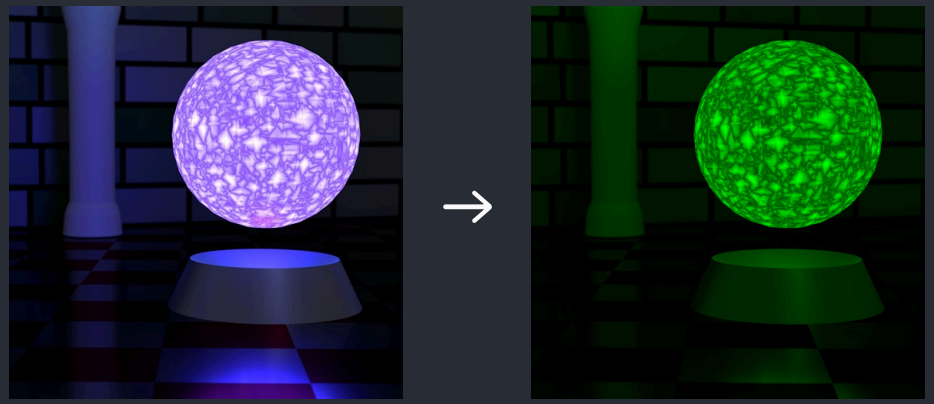
- メモリ上にある画像を読み込んで表示
- AXI バスのマスター側としてメモリを読み出す
  - 読み出しなので AR / R チャネルを使う
- のちに描画回路で再利用する
  - ちゃんと実装していないと終わる

AXI？

- **Advanced eXtensible Interface** の略
- 通信インタフェースを定めたもの
- マスターとスレーブの間でデータをやりとりできる

フィルタ回路

- メモリから読み出した画像から R/G/B の一部を抜き出し
- フィルタリング処理を回路レベルで実装
- フィルタ後の画像は AXI でメモリに書き込む
  - AW / W / B チャネルが登場



例）フィルタ処理で G 要素のみ通した結果

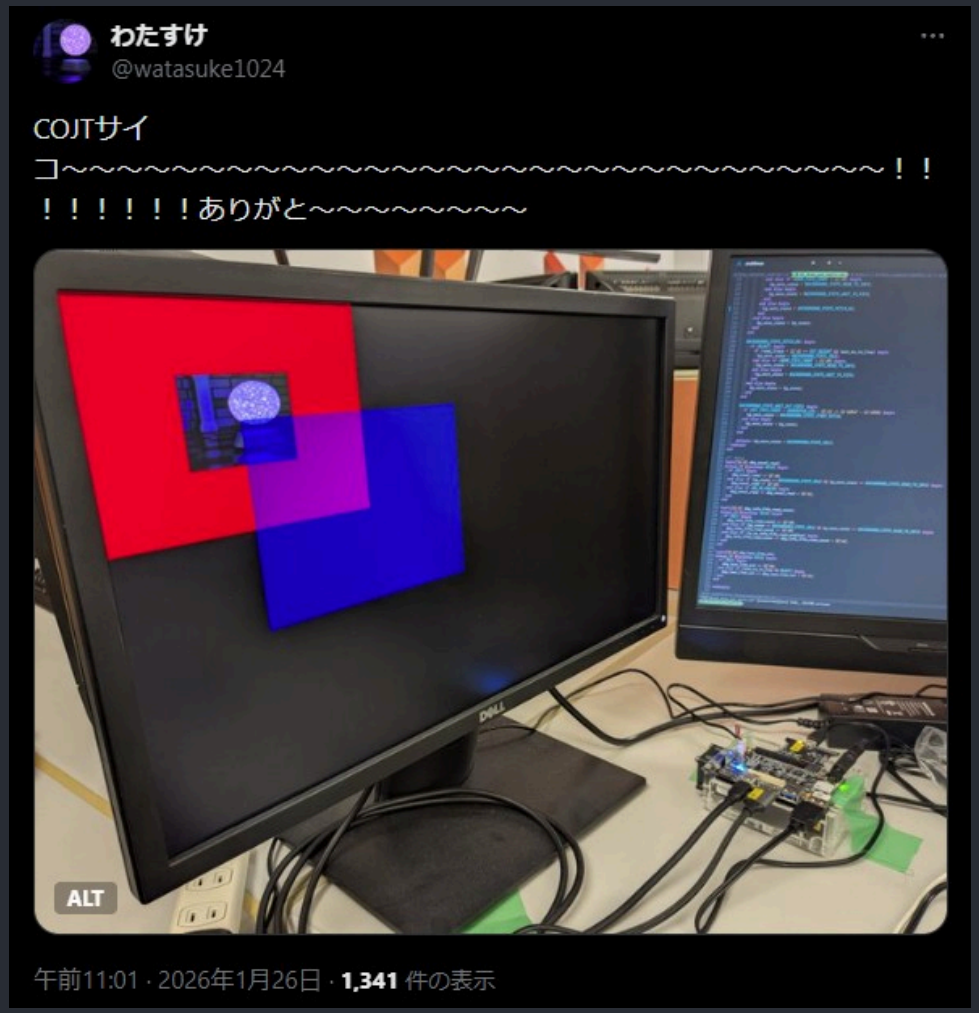
描画回路

- レジスタで描画コマンドを受け取る
- 座標やサイズを計算して、矩形あるいは画像を描画
  - AXI でメモリに書き込み
  - 書き込み結果は表示回路が出力
- 透明度（アルファブレンディング）も実装
- **実装が多くて難しい**（というか面倒）

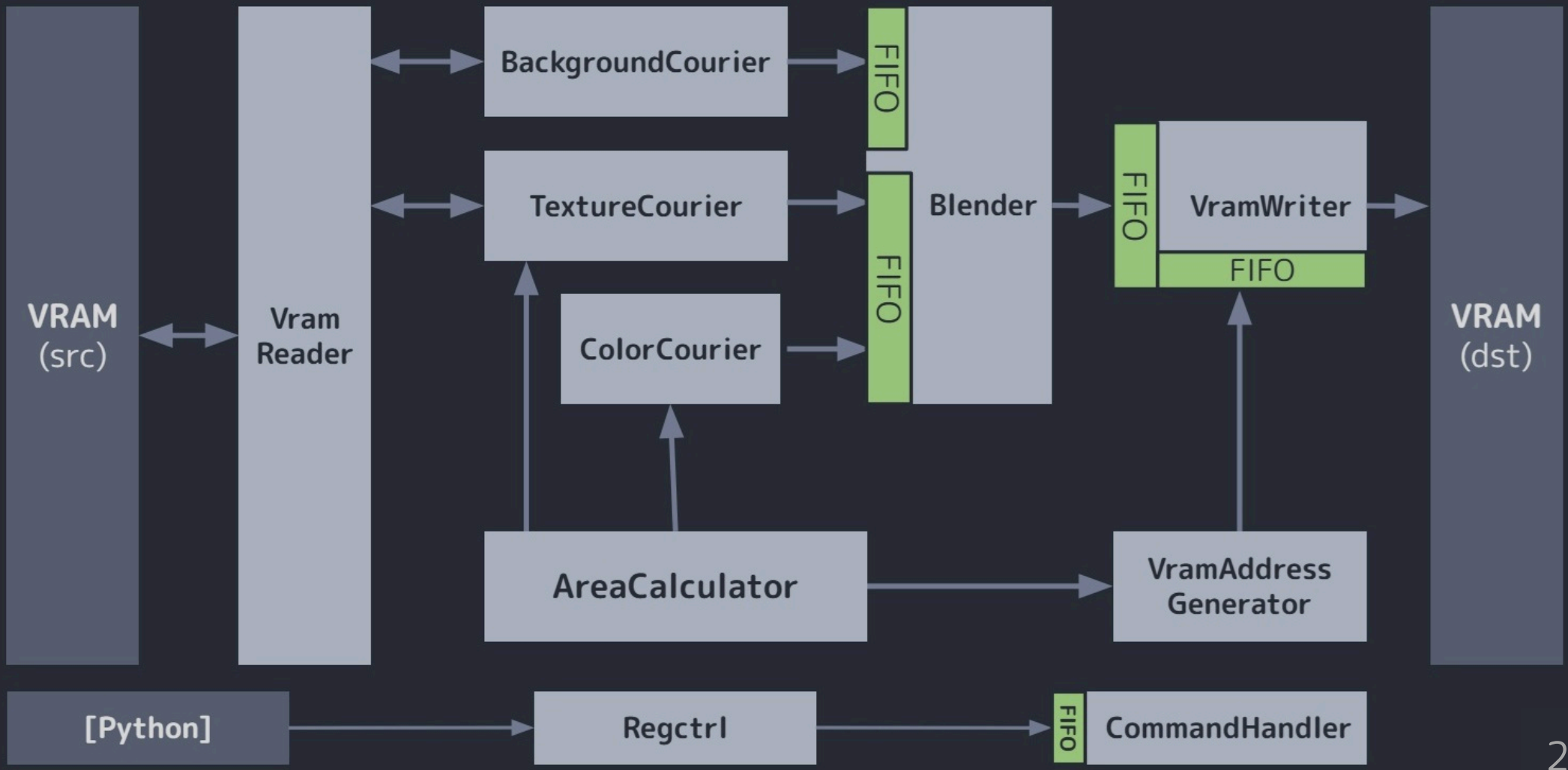
コマンド多すぎやねん（12 個！）

```
enum Commands {
  Nop, // 何もしません (00h) [3, 4]
  Eodl, // ディスプレイリスト終了 (0Fh) [3, 4]
  SetFrame { // フレームバッファ領域設定 (20h) [1, 4]
     frame_addr: u32, // フレームバッファの開始アドレス (4の倍数のみ) [1]
     width:      u11, // 水平サイズ (符号なし整数 11bit) [1]
     height:     u11, // 垂直サイズ (符号なし整数 11bit) [1]
   },
  SetDrawArea { // 描画領域設定 (21h) [4, 5]
     pos_x: u11, // フレームバッファ内での水平位置 (符号なし整数 11bit) [5, 6]
     pos_y: u11, // フレームバッファ内での垂直位置 (符号なし整数 11bit) [5, 6]
     siz_x: u11, // 描画領域の水平サイズ (符号無し整数 11bit) [5, 6]
     siz_y: u11, // 描画領域の垂直サイズ (符号無し整数 11bit) [5, 6]
  },

  SetTexture { // テクスチャ画像設定 (22h) [4, 6]
     fmt:          u1, // カラーフォーマット (0: ARGB8888, 1: RGB888) [6]
     texture_addr: u32, // テクスチャ画像のVRAM上での位置 (4の倍数のみ) [6]
  },
  SetFColor { // 描画カラー設定 (23h) [4]
     a: u8, // アルファ値 (8bit)
     r: u8, // 赤値 (8bit)
     g: u8, // 緑値 (8bit)
     b: u8, // 青値 (8bit)
  },
  SetStMode { // テクスチャの透過モード設定 (30h) [4, 7]
     mode: u8, // 透過判定方法 (0: 透過色無効, 1: 透過色有効) [8, 9]
     // 有効である場合、 ピクセル色 p が SCOLOR_L <= p <= SCOLOR_H であるピクセルは透過する
  },
  SetSColor { // テクスチャのステンシルカラー(透過色)設定 (31h) [4, 8]
     mask: u8, // 透過判定時のチャンネル無視マスク (4bit) [9-11]
     s_color_l_a: u8, // ステンシルカラー下限 A (ARGB8888フォーマット) [8, 9, 11]
     s_color_l_r: u8, // ステンシルカラー下限 R [8, 9, 11]
     s_color_l_g: u8, // ステンシルカラー下限 G [8, 9, 11]
     s_color_l_b: u8, // ステンシルカラー下限 B [8, 9, 11]

     s_color_h_a: u8, // ステンシルカラー上限 A (ARGB8888フォーマット) [8, 9, 11]
     s_color_h_r: u8, // ステンシルカラー上限 R [8, 9, 11]
     s_color_h_g: u8, // ステンシルカラー上限 G [8, 9, 11]
     s_color_h_b: u8, // ステンシルカラー上限 B [8, 9, 11]
  },
  SetBlendOff, // 画素間演算モードをOFFに設定 (32h) [4, 10]
  SetBlendAlpha { // 画素間演算モード設定 (33h) [4, 12]
     // 演算式: ((A-B)*C)/255 + (D*E)/255 [12]
     a_sel:       u8, // Aに入る式のセレクタ (3bit) [12, 13]
     b_sel:       u8, // Bに入る式のセレクタ (3bit) [12, 13]
     c_sel:       u8, // Cに入る式のセレクタ (3bit) [12, 13]
     d_sel:       u8, // Dに入る式のセレクタ (3bit) [12, 13]
     e_sel:       u8, // Eに入る式のセレクタ (3bit) [12, 13]
     src_const_a: u8, // ソース画像の固定α値 SRCCA [12, 13]
     coef0_a:     u8, // 固定カラー 0 (COEF0.A) [11-13]
     coef0_r:     u8, // 固定カラー 0 (COEF0.R) [11-13]
     coef0_g:     u8, // 固定カラー 0 (COEF0.G) [11-13]
     coef0_b:     u8, // 固定カラー 0 (COEF0.B) [11-13]
     coef1_a:     u8, // 固定カラー 1 (COEF1.A) [11-13]
     coef1_r:     u8, // 固定カラー 1 (COEF1.R) [11-13]
     coef1_g:     u8, // 固定カラー 1 (COEF1.G) [11-13]
     coef1_b:     u8, // 固定カラー 1 (COEF1.B) [11-13]
  },

  PatBlt { // 描画カラーで矩形を塗りつぶします (81h) [2, 4]
     dpos_x: s12, // 矩形の左上X座標 (符号付き整数 12bit) [2]
     dpos_y: s12, // 矩形の左上Y座標 (符号付き整数 12bit) [2]
     dsiz_x: u11, // 矩形の幅 (符号無し整数 11bit) [2]
     dsiz_y: u11, // 矩形の高さ (符号無し整数 11bit) [2]
  },
  BitBlt { // テクスチャ画像を転送します (82h) [4, 14]
     dpos_x: s12, // 転送先矩形の左上X座標 (符号付き整数 12bit) [14, 15]
     dpos_y: s12, // 転送先矩形の左上Y座標 (符号付き整数 12bit) [15]
     dsiz_x: u11, // 転送先矩形の幅 (符号無し整数 11bit) [15]
     dsiz_y: u11, // 転送先矩形の高さ (符号無し整数 11bit) [15]
     spos_x: s12, // テクスチャの転送開始X座標 (オフセット, 符号付き整数 12bit) [15]
     spos_y: s12, // テクスチャの転送開始Y座標 (オフセット, 符号付き整数 12bit) [15]
  },
}
```

多すぎて訳がわからなくなったので、Notion に Rust 風の構文でまとめなおした


わたすけ
@watasuke1024
COJTサイコ～～～

描画回路


VRAM
(src)
Vram
Reader
BackgroundCourier
TextureCourier
ColorCourier
FIFO
FIFO
Blender
FIFO
VramWriter
FIFO
VRAM
(dst)
AreaCalculator
VramAddress
Generator
[Python]
Regctrl
FIFO
CommandHandler

自由課題

なんかやれ！

はい……

（今まで作った回路を使って任意のものを作成する課題）

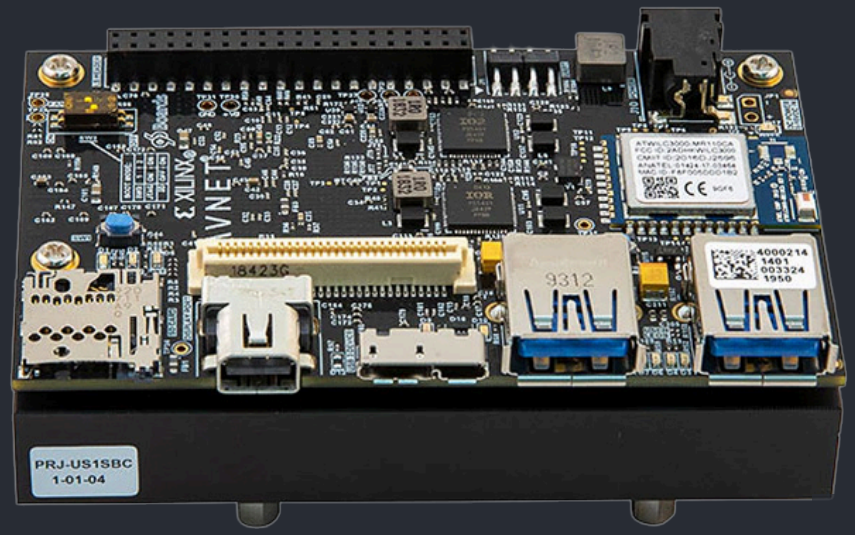
目次

1. つくったもの
2. COJT とは
3. COJT でのあゆみ
4. **「自由課題」のあゆみ**

COJT で使っていた FPGA ボード

Ultra96-V2

- CPU (64bit Arm) と FPGA の双方が乗っている
- Linux が動く
- FPGA は Linux から Python で制御



自由課題

- 描画回路は**位置を指定して画像を描画**できる
- Wayland はクライアントから**ウィンドウの画像**を受け取る
- FPGA のそばでは **Linux が動いている**

自由課題

- 描画回路は**位置を指定して画像を描画**できる
- Wayland はクライアントから**ウィンドウの画像**を受け取る
- FPGA のそばでは **Linux が動いている**

→ Wayland コンポジタのレンダリング、
FPGA にやらせたらおもろいやろな……笑

目標

- Wayland コンポジタを作る
- そこでブラウザを開いて Google スライドを見せる
- **プレゼン兼デモをぜんぶ FPGA でやる！**

Wayland コンポジタ と わたすけ

- 実は Wayland コンポジタづくりは **3 回目**
- 1 回目：**Zwin project**
  - https://zwin.dev
  - 2022 年度 未踏アドバンスト事業
  - Meta Quest 2 で動く VR デスクトップ環境

Wayland コンポジタ と わたすけ

- 2 回目：**yaza**
  - 高専の卒業研究、未踏 Adv の延長線 (?)
  - Android (Termux) で動き、レンダリング機能をもたない
  - Unity 製アプリと gRPC で通信して
    XREAL Air 2 (AR グラス) にデスクトップ環境を提供
    - XREAL SDK は Unity のみ！

Wayland コンポジタ と わたすけ

- XREAL に表示している映像を WebSocket で転送する機能を実装した
- 卒研発表でこれを使ってデモをしようとしたら本番でフリーズして死んだ
  - 発表練習で見たことがなかったバグを踏んだ
- **COJT の成果発表会でリベンジしたい !!**
  - 作業時期も 1 年前の卒研と被ってててすごい

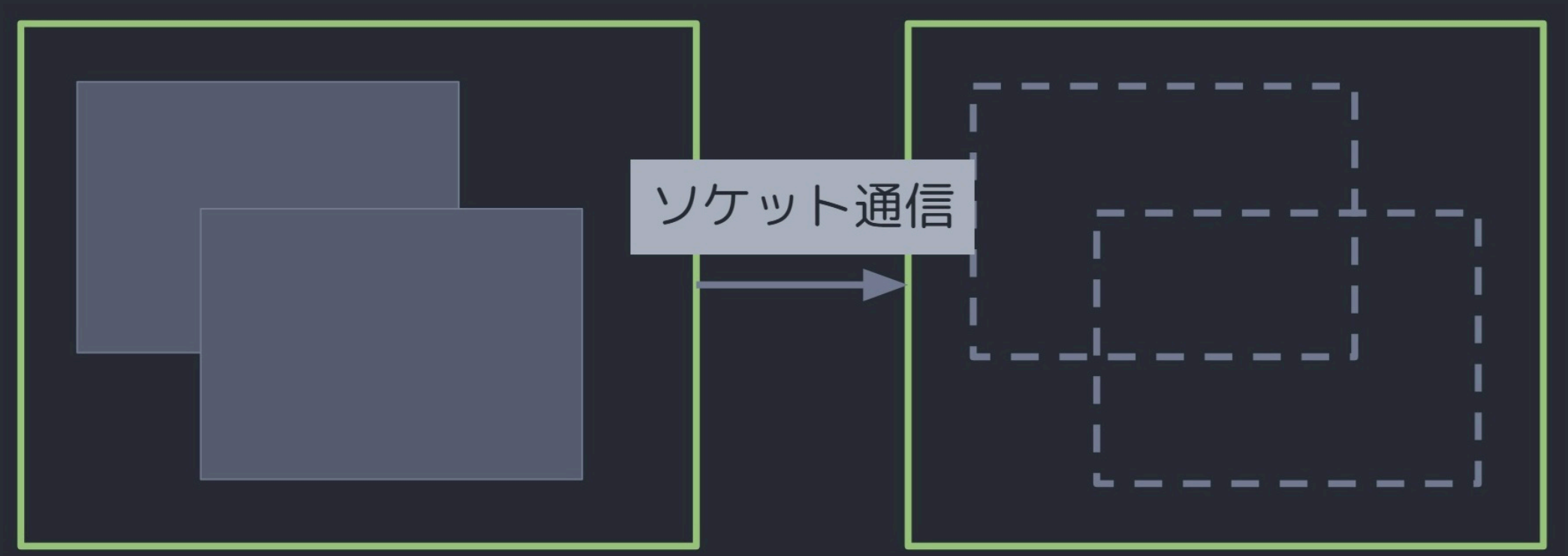
Wayland のしくみ

- クライアントは **surface** を作る
  - 0 個以上の出力装置に表示される可能性のある四角い領域
- また、コンポジタとの共有メモリ **“buffer”** も作成する
- **surface に buffer をアタッチ**することで、ウィンドウの画像を指定できる
- `commit()` を呼ぶことで、画像を更新したよ〜と伝える

実装方針

1. **Wayland コンポジタ**：クライアントから buffer を受け取る
2. **FPGA**：buffer を AXI で読み取る
3. 読み取ったものを画像として描画

しくみ



Wayland コンポジタ
C++

描画回路
Python & System Verilog

```
void Surface::commit() {
  if (this->buffer_.has_value()) {
    auto buffer = this->buffer_.value();
    struct wl_shm_buffer* shm_buffer = wl_shm_buffer_get(buffer);
    server::get().platform->commit_buffer(this->id, buffer);
  }
  // ...
}
```

commit() の処理 (1)

```
void FpgaPlatform::commit_buffer(
  const core::Surface::Id& id, wl_resource* buffer) {
  constexpr uint8_t kCommitCommand = 0x10;

  auto buf_addr = (uint64_t)wl_shm_buffer_get_data(shm_buffer);
  send(this->socket_fd_, &kCommitCommand, 1, 0); // !

  wl_shm_buffer_begin_access(shm_buffer);
  send(this->socket_fd_, &buf_addr, 8, 0);
  wl_shm_buffer_end_access(shm_buffer);
}
```

commit() の処理 (2)

```python
surfaces: dict[int, Surface] = {}

def handle_event(self, conn, data):
  match data:
    # Surface.commit
    case b'\x10':
      surface_id = int.from_bytes(conn.recv(4), 'little')
      width      = int.from_bytes(conn.recv(4), 'little')
      # ...
      if self.surfaces.get(surface_id) == None:
        self.surfaces[surface_id] = Surface()
      self.surfaces[surface_id].commit(width, height, fmt, addr)
```

受け取り手 (1)

```
def commit(self, width, height, fmt, addr):
  if self.buffer.size == 0:
    self.width  = width
    self.height = height
    # !!
    self.buffer = allocate(
      shape = (width, height, self.pixel_size), dtype='u1')

  ctypes.memmove(self.buffer.ctypes.data, ctypes.c_void_p(addr),
    width * height * self.pixel_size)

  self.buffer.flush()
```

受け取り手 (2)

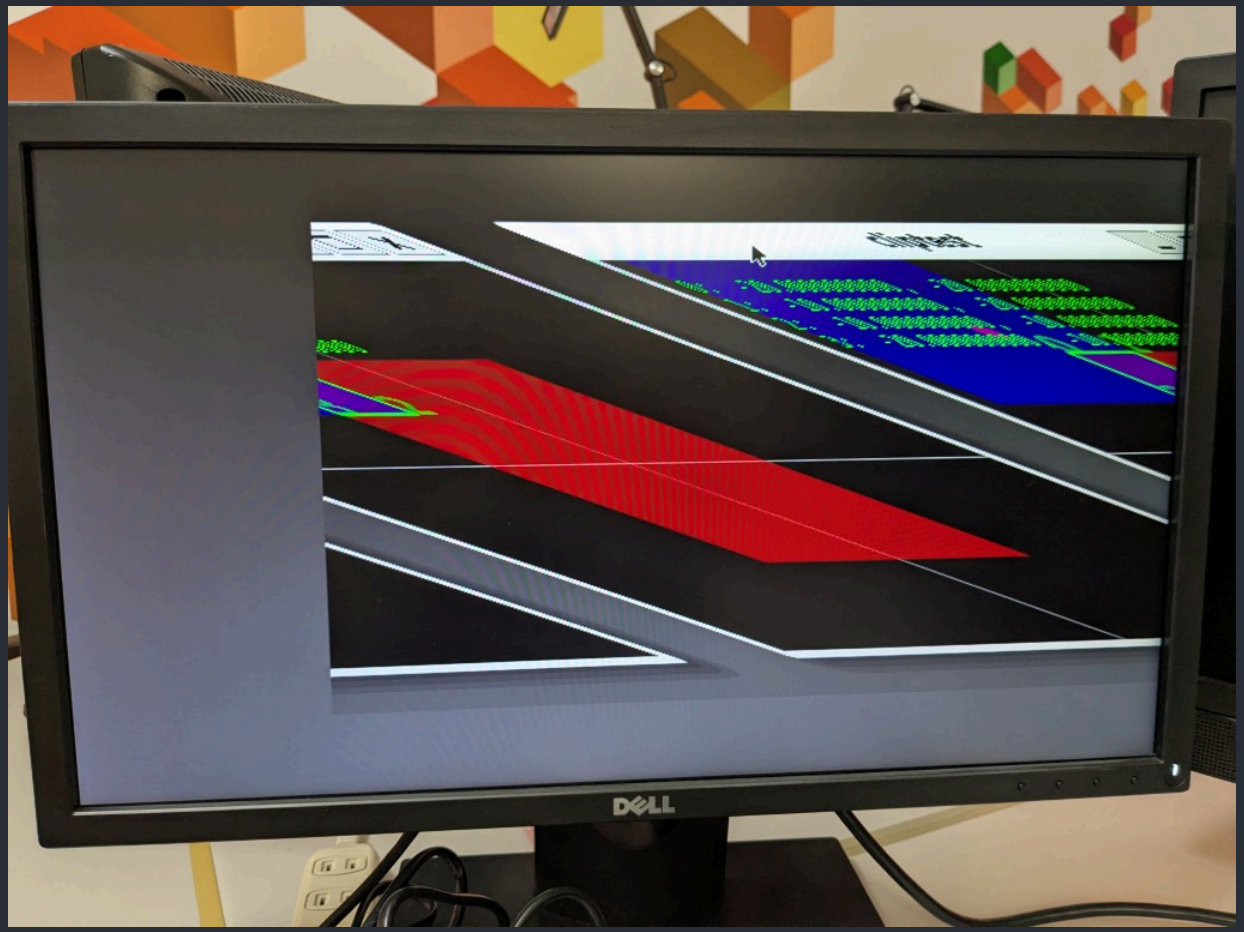
コンポジタ動作のようす

マウス入力

コンポジタ動作のようす

アルファブレンド

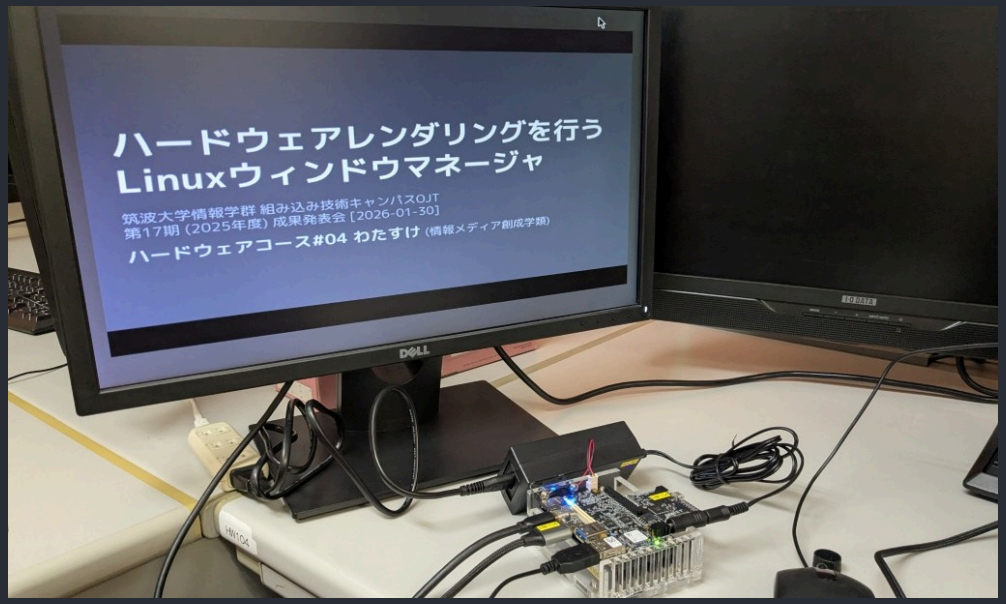
コンポジタ動作のようす



ウィンドウが画面の**下以外**にはみ出ると、崩壊

デモ

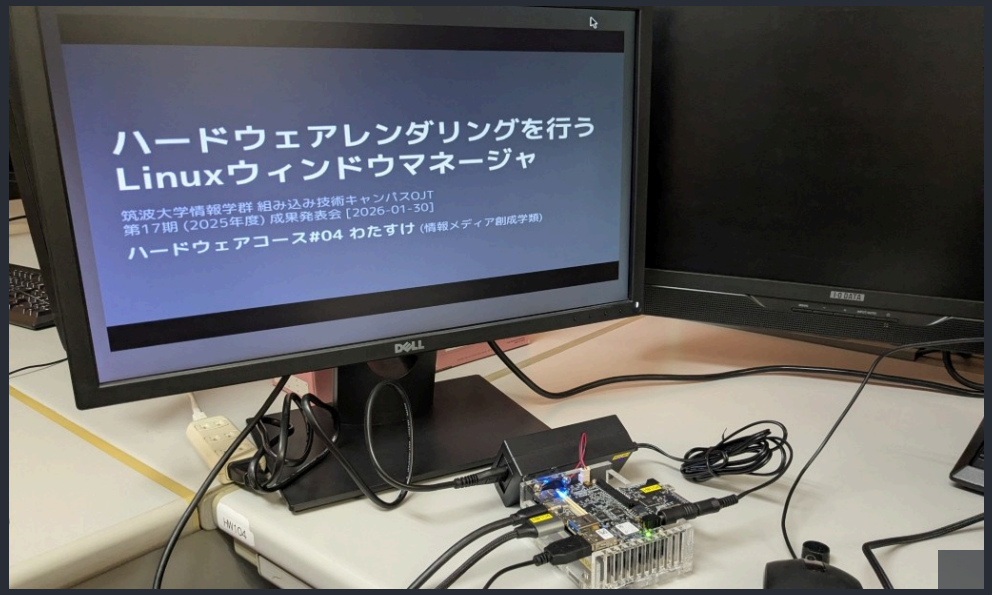
- 発表会の直前に Web ブラウザの起動に成功
- ほぼ **バグなしでデモ完走！**
  - 超うれし〜



おわり

- 筑波大学で COJT というおもしろ授業を受けた
- FPGA にレンダリングさせる Wayland コンポジタを作った
- その中で Web ブラウザを開いて
  Google スライドで発表した
- COJT ありがと〜

←作成したコンポジタ



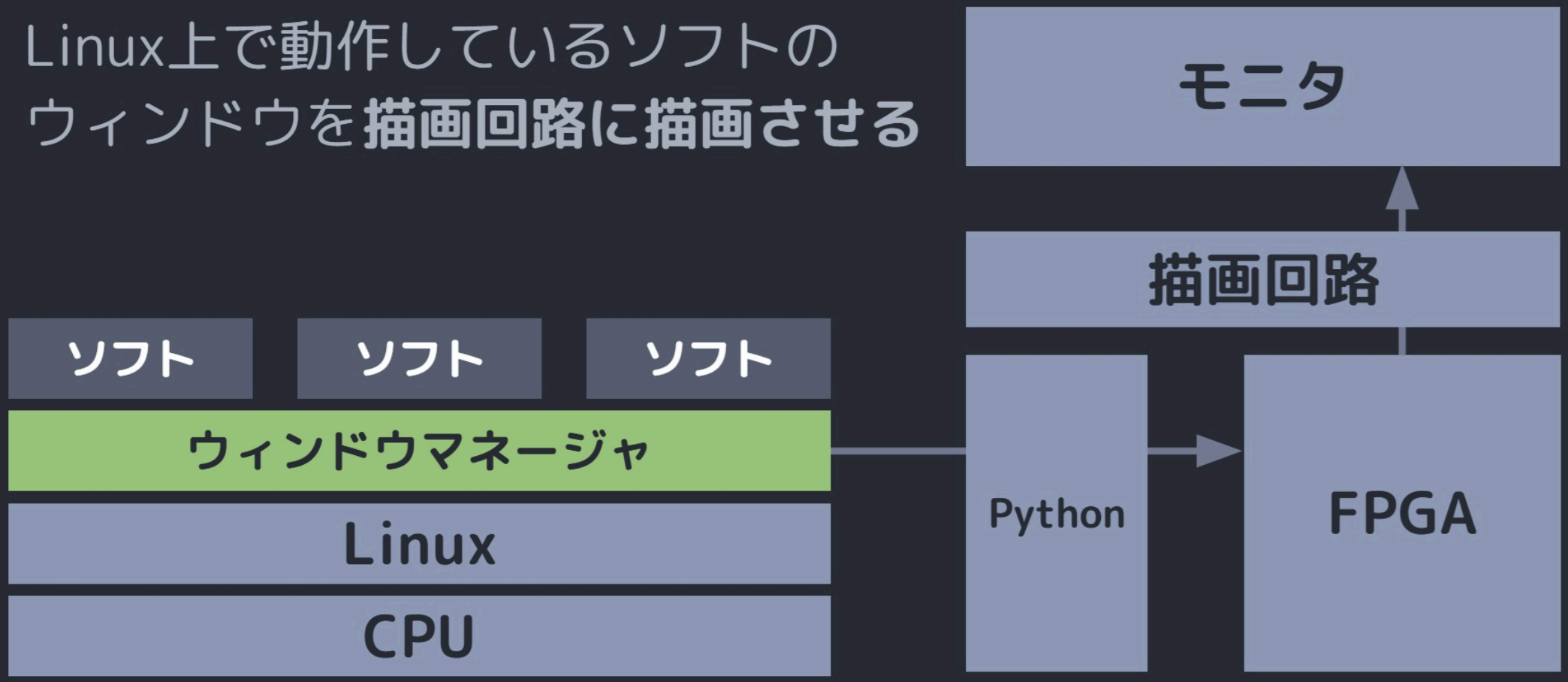
なんでコンポジタの buffer を普通に読んでるの？

- Python がコンポジタを別スレッドとして起動するため、直接メモリを読める
  - もっとうまいやり方はいくらでもあるだろうが、実装速度を優先した

```
def start_compositor(self):
  lib = ctypes.cdll.LoadLibrary('fpga-build/libcojt-compositor.so')
  thread = threading.Thread(target = lib.main)
  thread.start()
```

未実装の機能

- キーボード入力
  - 頑張ったけど発表会には間に合わなかった
- ポップアップ（右クリックメニュー）
- コピー & ペースト
- ドラッグ & ドロップ

全体像

Linux上で動作しているソフトの
ウィンドウを**描画回路に描画させる**